CommonsMesh — NGI Zero Commons Fund Application Attachment

Supplementary Technical Proposal and Milestone Plan
Submitted as an attachment to the NGI Zero Commons Fund application.

Part A: Strategic Alignment with the Next Generation Internet Vision

When I was doing architectural field research in the informal settlements of Port Moresby, Papua New Guinea, I saw firsthand how communities survive on the margins. They don't rely on state infrastructure; they rely on dense, informal networks of mutual aid. They share tools, labor, and information. But coordinating this sharing is incredibly difficult, especially when internet access is spotty and people are time-poor.

Currently, these communities are forced to use corporate platforms like WhatsApp or Facebook Groups to organize. This is a mismatch for two reasons. First, chat apps are designed for conversation, not coordination. Important needs get buried. Second, these platforms extract data and trap vulnerable populations in surveillance capitalism.

CommonsMesh is my attempt to build a technical alternative that aligns directly with the Next Generation Internet's vision of a human-centric, trustworthy web. It provides a local-first, privacy-preserving coordination layer. Because it's built on libp2p, it doesn't need a central server. The data stays on the users' devices.

By moving away from a "social feed" model to a "structured community graph" model, CommonsMesh turns chaotic group chats into actionable mutual aid. And by embedding a lightweight LLM directly on the phone to help match needs and resources, it provides the benefits of AI without sending community data to a cloud provider. This is exactly the kind of open, decentralized public good that the NGI initiative aims to support.

Part B: Detailed Technical Architecture

I've spent the last five years writing highly optimized code for smart glasses—edge devices with severe constraints on battery, memory, and connectivity. I'm applying those exact same optimization techniques to CommonsMesh.

B.1 Protocol Layer

The core of CommonsMesh isn't the UI; it's the protocol. I treat every community action as a cryptographically signed event envelope.

When a user interacts with the app, they generate an event (like `graph.delta` to declare a need, or `project.motion` to propose a group action). This event is signed with an Ed25519 key generated on their device. I use canonical JSON serialization and a SHA-256 hash to ensure the payload can't be tampered with. Each event links to the previous one via a `chain_hash`, creating a tamper-evident, append-only log for every user.

To prevent Sybil attacks (spamming the network with fake accounts) in a serverless environment, I've designed a multi-layered validator pipeline. It checks temporal windows (rejecting events older than 5 minutes to prevent replay attacks) and enforces a local trust-scoring system. High-impact actions require a minimum trust score, which is built up through social proofs (`trust.attest` events) from existing community members.

B.2 Network Layer (libp2p)

The network layer is built on libp2p. I'm configuring the node factory to handle the messy reality of mobile networks: using TCP and WebSockets for transport, mDNS for discovering peers on the same local network, and GossipSub for broadcasting events.

The hardest part of mobile P2P is state synchronization when devices go offline and come back online. I'm implementing an incremental sync protocol using Bloom filters. When two phones connect, they exchange tiny Bloom filters representing their known event hashes. They calculate the difference and only request the missing chunks. This keeps bandwidth usage extremely low, which is vital for users on prepaid data plans.

B.3 Local Graph Database

All data is stored locally using SQLite (via `expo-sqlite` on the React Native side). The schema is simple but powerful. There's an append-only `events` table that stores the raw cryptographic envelopes. From that, the app derives `graph_nodes` and `graph_edges` tables, which represent the current state of the community (who needs what, who is working on what project).

This local graph is what the UI queries, and it's what feeds into the on-device AI. Because it's local, the app works perfectly fine offline; it just syncs the state changes the next time it finds a peer.

B.4 On-Device LLM Integration

This is where my edge computing background really comes into play. I'm integrating `llama.cpp` into the React Native app using `llama.rn` to run GGUF-format models (like Qwen2.5 1B) locally.

The matching engine runs in two stages to save battery. The first stage is a deterministic, rule-based engine that looks for exact keyword matches in the local SQLite graph. If it finds a match, it stops there.

The second stage only wakes up the LLM when the phone is idle or charging. It feeds the LLM a JSON summary of the unmatched needs and resources. The prompt asks the model to find semantic matches (e.g., realizing that someone offering "truck space" can help someone who needs "moving boxes") or to spot aggregate patterns (e.g., noticing that five people all need the same building material, and proposing a bulk purchase motion). The LLM outputs structured JSON, which the app turns into actionable UI elements.

Part C: Milestone Plan

I've structured this as a 9-month development cycle. I'm building this independently, and each milestone results in a concrete, open-source deliverable.

Milestone	Duration	Deliverables	Verification
M1: Protocol & Security	Months 1–2	Finalized event schema, the validator pipeline (anti-Sybil, replay protection), and the capability token system.	Code pushed to GitHub; test suite passing.
M2: P2P Network Sync	Months 3–4	libp2p node setup for mobile, Bloom filter sync implementation.	Code pushed to GitHub; demo video showing two nodes syncing.
M3: Local AI Integration	Months 5–6	`llama.rn` integrated, two-stage matching engine built, prompts optimized for 1B models.	Code pushed to GitHub; benchmark report on inference time/battery usage.
M4: Mobile App Build	Months 7–8	React Native (Expo) app completed, UI wired to the local graph DB.	Code pushed to GitHub; end-to-end test video on physical Android/iOS devices.
M5: Pilot & Docs	Month 9	Run a pilot with a local mutual aid group, write the protocol specs, prep for security audit.	Pilot report and technical documentation published.

Part D: Sustainability and Long-Term Impact

I'm releasing everything under the AGPL-3.0 license. This ensures the codebase remains a digital commons. If a commercial entity wants to use the protocol, they have to contribute their improvements back.

Because the architecture is local-first and serverless, there are no ongoing cloud hosting costs to worry about. The community sustains the network simply by running the app on their phones.

In the long run, I want the CommonsMesh protocol to become a standard building block. Just like ActivityPub standardized federated social media, I hope this protocol can standardize decentralized community coordination. If other projects adopt the event schema, we could see interoperable mutual aid networks that don't rely on any single app or platform.

Part E: Relation to Existing NGI Zero Projects

I'm heavily leveraging the libp2p ecosystem, which has received NGI support in the past. While projects like Secure Scuttlebutt (which I admire) focus on federated social feeds, CommonsMesh is focused strictly on structured coordination and state changes. It fills a different niche: it's not for chatting; it's for getting things done.

I'm very open to collaborating with other NGI Zero grantees, especially anyone working on local-first data sync or decentralized identity, as those areas overlap heavily with what I'm building.

Part F: Open Source Commitment

I commit to releasing all software, documentation, and protocol specifications produced under this grant under the **AGPL-3.0-only** license.

The repository is already public at: https://github.com/e982happy/CommonsMesh

I will maintain this repository, document the code thoroughly, and ensure that the work funded by NGI Zero is easily accessible and reusable by the broader open-source community.